



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



Department of Computer Science and Engineering

ALGORITHMIC PROBLEM-SOLVING HACKATHON (APSH 2026)

SMART INVENTORY MANAGEMENT

Field	Details
Project Title	Smart Inventory Management
Team Name / Team No.	20
Year / Semester / Section	II Year B.Tech
Algorithm Paradigm Used	Dynamic Programming – 0/1 Knapsack
Programming Language / Tools	HTML, CSS, JavaScript, Web Browser
Faculty Mentor	Sujata Mam

Team Members

S.No	Name of the Student	Roll No.	Role
1	DEVJI BHAGYASREE	25MVC SER0030	Team Lead
2	JUKANTI BHAVYA	25MVC SER0051	Member
3	NARI SREEJA	25MVC SER0066	Member
4	POCHAMAINA AKSHITA	25MVC SER0072	Member

1. Problem Statement

Problem: Select products to stock within a limited storage capacity and maximize the total expected benefit.

Item	Details
Input	N, storage capacity C, storage requirement $W[i]$, expected benefit $B[i]$
Output	Maximum benefit, selected products, total storage used, storage utilization
Selection	Each product can be selected once or not selected
Constraint	Total selected storage must be $\leq C$
Numerical limits	Not given in the original statement; practical limits are used for implementation/testing

2. Objectives

- Select products within the available storage capacity.
- Maximize total expected benefit.
- Use Dynamic Programming to solve the 0/1 Knapsack problem.
- Show the selected products and storage utilization in a web interface.

3. Proposed Approach

Item	Details
Problem Type	Optimization / Maximization
Algorithm	0/1 Knapsack
Paradigm	Dynamic Programming
Decision	Include or exclude each product
Reason	The problem has overlapping subproblems and optimal substructure

Alternatives considered: Greedy, Backtracking and Branch & Bound. Greedy is not guaranteed to give the exact optimum for a general 0/1 Knapsack problem.

4. Algorithm / Pseudocode

- Read N and C.
- Read storage requirement and benefit for each product.
- Create a DP table of size $(N+1) \times (C+1)$, initialized to 0.
- For every product and capacity, compare excluding and including the product.
- Store the larger value in the DP table.
- $DP[N][C]$ gives the maximum benefit.
- Trace the table backwards to find the selected products.
- Calculate total storage and utilization, then display the result.

Pseudocode

```
START
Read N, C
Read W[1..N] and B[1..N]

Create DP[0..N][0..C] = 0

FOR i = 1 TO N
  FOR c = 0 TO C
    DP[i][c] = DP[i-1][c]
    IF W[i] <= c THEN
      include = B[i] + DP[i-1][c-W[i]]
      DP[i][c] = MAX(DP[i][c], include)
    END IF
  END FOR
END FOR
END FOR
```

```

maxBenefit = DP[N][C]
i = N, c = C
WHILE i > 0
    IF DP[i][c] != DP[i-1][c] THEN
        Select product i
        c = c - W[i]
    END IF
    i = i - 1
END WHILE

Display selected products, maxBenefit, total storage and utilization
END

```

5. Complexity Analysis

The DP table contains $N \times C$ states. Each state takes constant time to calculate.

Complexity	Best Case	Average Case	Worst Case
Time	$O(NC)$	$O(NC)$	$O(NC)$
Space	$O(NC)$	$O(NC)$	$O(NC)$

6. Implementation

Component	Used
Frontend	HTML
Styling	CSS
Logic	JavaScript
Algorithm	0/1 Knapsack using Dynamic Programming
Data Structures	Arrays, objects, 2D DP table
Platform	Web browser

Main functions:

- addProduct() – adds a product row.
- removeProduct() – removes a product row.
- getProducts() – reads product details.
- optimizeInventory() – validates input, runs DP and finds the selected products.
- displayResults() – shows maximum benefit, storage used and utilization.
- displaySelectedProducts() – shows selected product cards.
- displayBenefitChart() – shows benefit visualization.

Project files: index.html, style.css, script.js

7. Results and Test Cases

Test Case 1 – Sample Inventory

Product	Storage	Benefit
Laptop	5	60
Phone	3	40
Tablet	4	50
Smart Watch	2	20

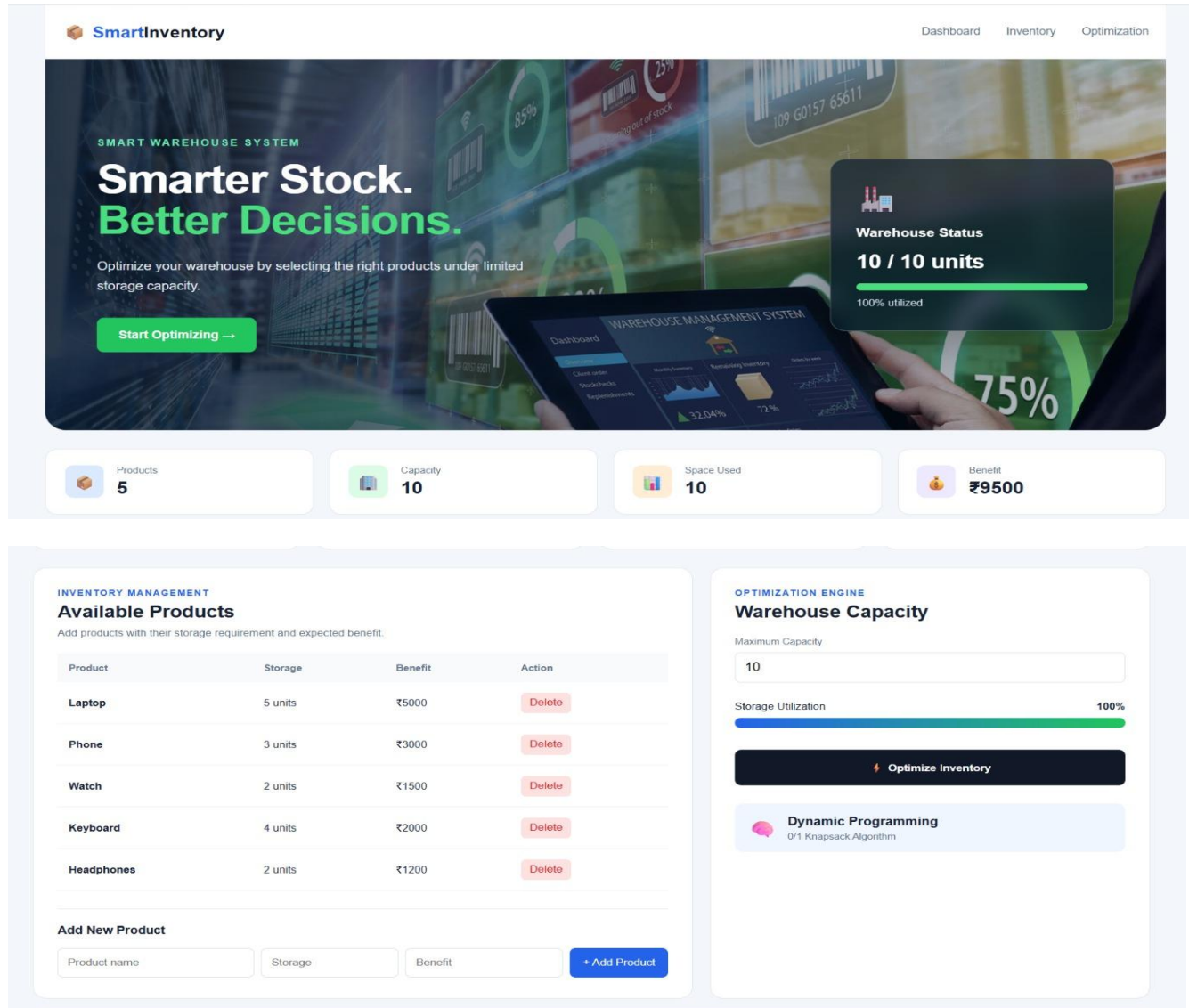
Storage Capacity: 10

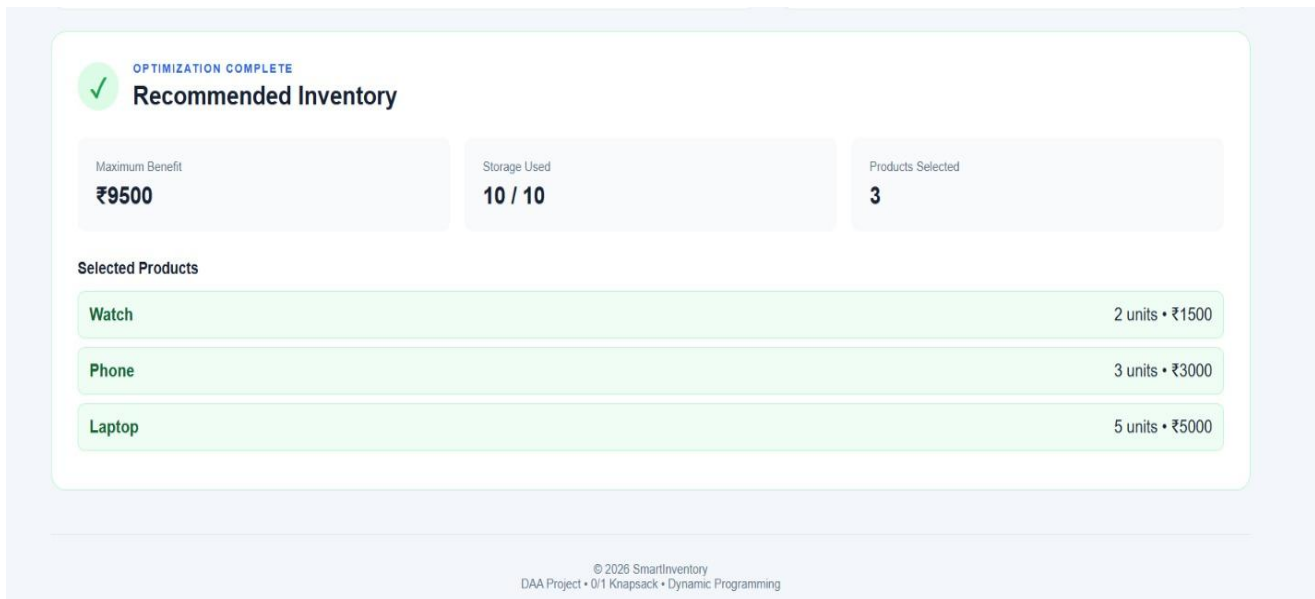
Output	Result
Maximum Benefit	110
Selected Products	Laptop, Tablet
Total Storage Used	9
Storage Utilization	90%

Test Case 2

Input	Output
Capacity = 5; P1=(2,6), P2=(3,10), P3=(4,12)	Maximum Benefit = 16; Selected = P1, P2; Storage = 5

Working Model: The user enters capacity and product details. The system runs the DP algorithm and displays the selected products, maximum benefit, storage used, utilization and benefit char





8. Conclusion and Future Scope

Conclusion

- The project implements Smart Inventory Management using 0/1 Knapsack and Dynamic Programming.
- The system finds the product combination with maximum benefit without exceeding capacity.
- The web interface provides the result in an easy-to-understand form.

Future Scope

- Connect with an inventory database.
- Use sales history and demand data.
- Add budget, expiry date and category constraints.
- Support multiple storage locations.
- Add demand forecasting.

9. References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [2] S. S. Skiena, The Algorithm Design Manual, 3rd ed. Cham, Switzerland: Springer, 2020.
- [3] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, Data Structures and Algorithms in Python. Hoboken, NJ, USA: Wiley, 2013.
- [4] MDN Web Docs, "JavaScript Guide."
- [5] MDN Web Docs, "HTML: HyperText Markup Language."
- [6] MDN Web Docs, "CSS: Cascading Style Sheets."

— End of Project Documentation —